Лекция 2. Логирование, конвейер обработки запросов, фильтры

1. Логирование

Логирование — это запись событий, происходящих в приложении во время его работы. В ASP.NET встроена система логирования, доступная через интерфейс `ILogger<T>`. Она подключается через DI-контейнер и не требует дополнительных пакетов для базового использования.

Параметр `T` — это класс, в котором используется логгер. Он нужен только для того, чтобы в логах было видно, из какого именно места пришла запись.

Получение логгера

Логгер внедряется через конструктор, как любая другая зависимость:

```
public class ProductsController : ControllerBase
{
    private readonly ILogger<ProductsController> _logger;

    public ProductsController(ILogger<ProductsController> logger)
    {
        _logger = logger;
    }

    [HttpGet]
    public IActionResult GetAll()
    {
        _logger.LogInformation("Запрошен список продуктов");
        return Ok();
    }
}
```

Уровни логирования

Каждая запись в лог имеет уровень, который отражает важность события. Уровни расположены по возрастанию серьёзности:

```
_logger.LogTrace("Детальная отладочная информация: значение переменной x =
{X}", x);
_logger.LogDebug("Отладочная информация: начало обработки заказа {OrderId}",
orderId);
_logger.LogInformation("Пользователь {UserId} успешно вошёл в систему",
userId);
_logger.LogWarning("Попытка обратиться к удалённому продукту {ProductId}",
productId);
_logger.LogError(ex, "Ошибка при сохранении продукта {ProductId}",
productId);
_logger.LogCritical("База данных недоступна, приложение не может продолжать
работу");
```

Практическое правило выбора уровня:

- `Trace` и `Debug` — детали, нужные только при активной отладке. В production обычно отключены.
- `Information` — штатные события: запрос обработан, пользователь авторизован, задача выполнена.
- `Warning` — что-то пошло не по плану, но приложение справилось. Стоит обратить внимание.
- `Error` — операция завершилась ошибкой. Требует разбора, но приложение продолжает работать.
- `Critical` — приложение не может продолжать нормальную работу. Требует немедленного вмешательства.

```
info: WebApplication1.Controllers.ValuesController[0]
      Информационное сообщение
warn: WebApplication1.Controllers.ValuesController[0]
      Предупреждение
fail: WebApplication1.Controllers.ValuesController[0]
      Сообщение об ошибке
crit: WebApplication1.Controllers.ValuesController[0]
      Сообщение о критической ошибке
```

Структурированное логирование

Важная особенность `ILogger` — поддержка структурированного логирования. Вместо конкатенации строк следует использовать именованные параметры в фигурных скобках:

```
// Плохо: строка просто склеивается, параметры теряются как отдельные поля
_logger.LogInformation("Создан продукт с id " + product.Id + " и именем " +
product.Name);

// Хорошо: ProductId и ProductName сохраняются как отдельные поля
// Это позволяет фильтровать и искать по ним в системах сбора логов
_logger.LogInformation(
    "Создан продукт {ProductId} с именем {ProductName}",
    product.Id,
    product.Name);
```

Настройка минимального уровня

Минимальный уровень логирования настраивается в `appsettings.json`. Записи с уровнем ниже указанного будут проигнорированы:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

Здесь `Default` задаёт уровень для всего приложения, а `Microsoft.AspNetCore` — отдельный уровень для внутренних компонентов ASP.NET, которые в противном случае генерируют большое количество служебных записей.

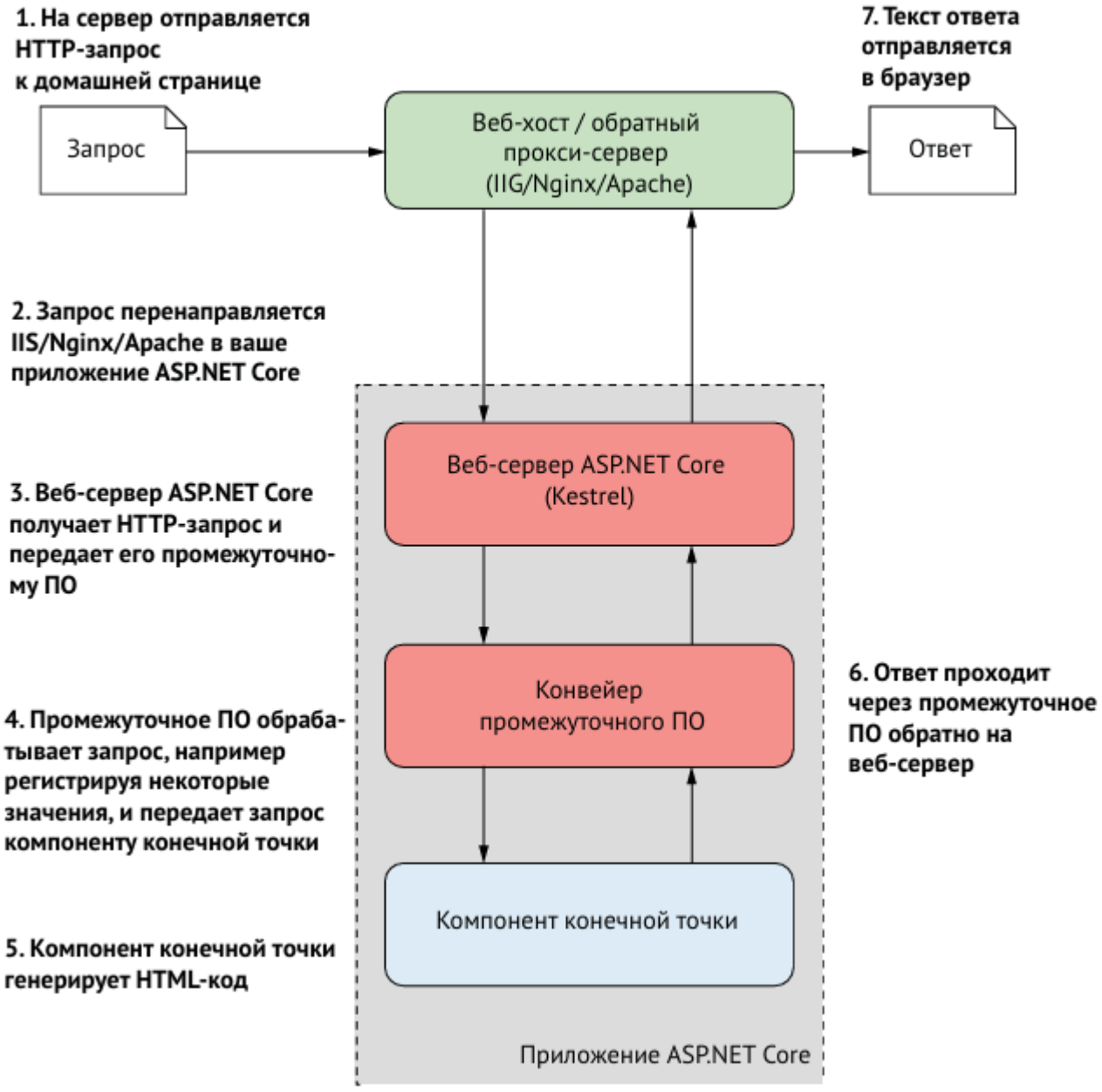
Как правило, в разработке используется уровень `Debug` или `Information`, а в production — `Warning` или `Error`, чтобы не перегружать систему лишними записями.

2. Конвейер промежуточного ПО (Middleware Pipeline)

В предыдущей лекции было рассмотрено общее устройство ASP.NET-приложения: как Kestrel принимает входящие соединения, как регистрируются контроллеры и каким образом DI-контейнер предоставляет зависимости. Однако между моментом, когда сервер получил HTTP-запрос, и моментом, когда выполняется метод контроллера, запрос проходит через несколько слоёв обработки.

Middleware — это компонент, который получает входящий HTTP-запрос, выполняет некоторую логику и принимает одно из двух решений:

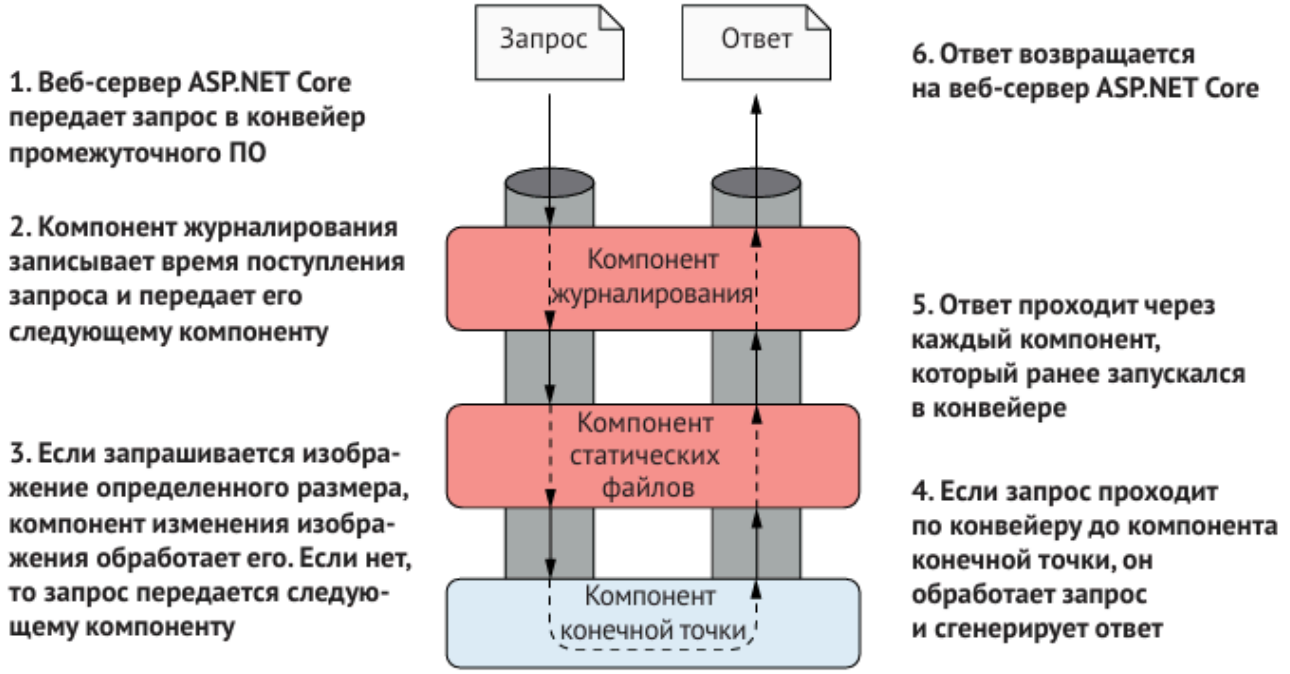
- Передать запрос дальше по конвейеру, вызвав следующий компонент через делегат `next`
- Завершить обработку самостоятельно, сформировав ответ и вернув его клиенту без вызова `next`.



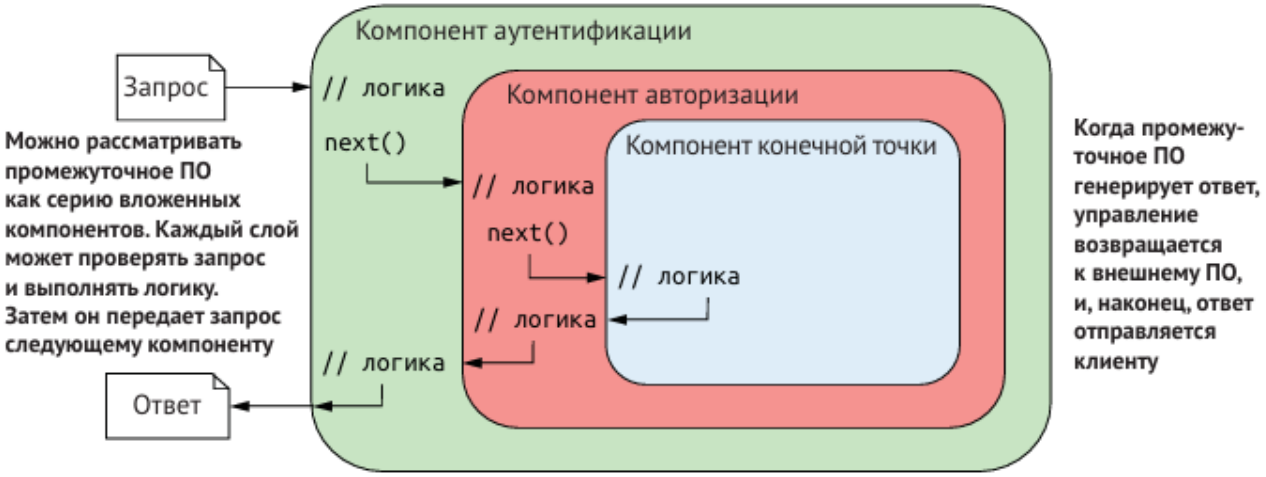
Конвейер строится как цепочка таких компонентов. Каждый из них оборачивает следующий:

- сначала выполняется код до вызова `next` (обработка запроса)
- затем вызывается следующий компонент
- после его завершения управление возвращается обратно, и выполняется код после `next` (обработка ответа).

При этом по всей цепочке от начала и до конца, а затем обратно, передаётся один и тот же экземпляр `HttpContext`. Этот объект инкапсулирует всю информацию о входящем HTTP-запросе (метод, путь, заголовки, тело) и позволяет формировать ответ, устанавливая статус-код, заголовки и содержимое. Именно через `HttpContext` middleware обмениваются данными: один компонент может записать в `Items` служебную информацию, а другой, находящийся дальше по конвейеру или на обратном пути, прочитает её.

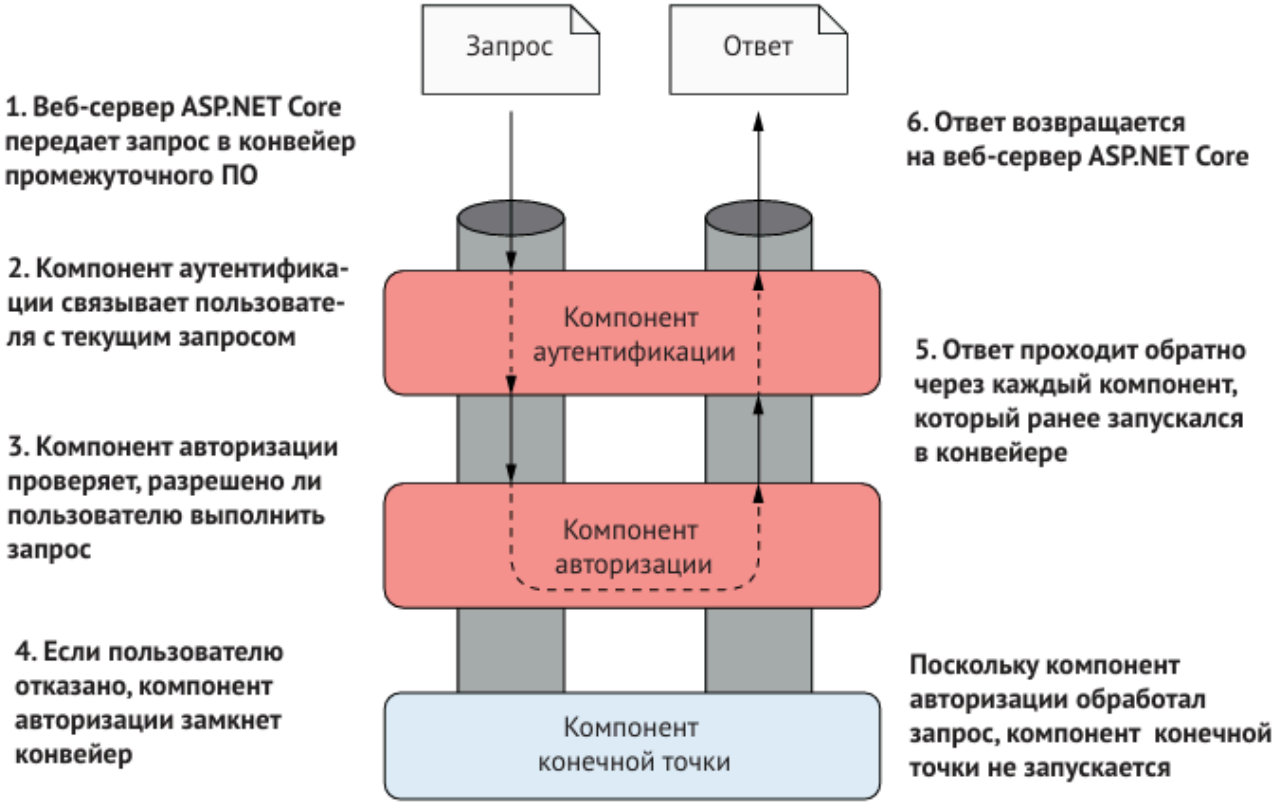


Визуально компоненты можно представить в виде вложенных друг в друга блоков (на подобие матрёшки или луковицы). Запрос проходит по всем этапам промежуточного ПО, доходит до конечной точки и идёт обратно. Внешний блок начинает работу первым и завершает последним.



Замыкание конвейера

Если на каком-либо этапе компонент решает не вызывать `next`, происходит **замыкание конвейера**. Это означает, что запрос не доходит до конечной точки и последующих middleware, а немедленно начинает движение в обратном направлении — к клиенту. Чаще всего замыкание происходит при обнаружении ошибки или при невыполнении условия доступа. Например, компонент авторизации может установить статус-код 401 или 403 и завершить обработку, не передавая запрос дальше. В этом случае все middleware, расположенные после текущего, не выполнятся вовсе, а управление вернётся к предыдущим компонентам, которые могут выполнить пост-обработку ответа на обратном пути.



Порядок регистрации

Конвейер формируется в `Program.cs` через методы объекта `app` типа `WebApplication`, название компонентов обычно начинается с `Use*`. Порядок вызовов имеет принципиальное значение: компоненты выполняются строго в том порядке, в котором они зарегистрированы.

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.UseExceptionHandler("/error"); // 1. Перехват необработанных исключений
app.UseHttpsRedirection();         // 2. Перенаправление HTTP → HTTPS
app.UseRouting();                  // 3. Определение маршрута
app.UseAuthentication();           // 4. Аутентификация (кто делает запрос?)
app.UseAuthorization();            // 5. Авторизация (имеет ли право?)
app.MapControllers();              // 6. Endpoint

app.Run();
```

Например, если поменять местами `UseAuthentication` и `UseAuthorization`, авторизация будет вызвана до того, как установлена личность пользователя, и все защищённые эндпоинты вернут 401.

Создание middleware

Иногда может понадобиться написать собственный middleware для сквозной функциональности: логирование времени запроса, добавление заголовков, трассировка. Что-то что должно происходить для каждого запроса вне зависимости от контроллера.

Есть два способа написать middleware.

Для простых случаев подойдет лямбда-функция прямо в `Program.cs`:

```
app.Use(async (context, next) =>
{
    // Код до следующего компонента
    Console.WriteLine($"→ {context.Request.Method} {context.Request.Path}");

    await next(context); // передать запрос дальше

    // Код после — ответ уже сформирован
    Console.WriteLine($"← {context.Response.StatusCode}");
});
```

`next` — это следующий middleware в цепочке. Вызвать `next` — значит передать запрос дальше. Не вызвать — значит остановить обработку и вернуть ответ самостоятельно (так работает, например, проверка авторизации).

Пример — проверка API-ключа, если его нет или он неправильный, конвейер прерывается:

```
app.Use(async (context, next) =>
{
    // Проверяем наличие API-ключа в заголовке
    if (!context.Request.Headers.TryGetValue("X-Api-Key", out var key)
        || key != "secret-key")
    {
        // Не вызываем next — запрос дальше не идёт
        context.Response.StatusCode = StatusCodes.Status401Unauthorized;
        await context.Response.WriteAsJsonAsync(new
        {
            error = "Отсутствует или неверный API-ключ"
        });
        return; // выходим из middleware
    }

    await next(context); // ключ верный — передаём дальше
});
```

Если нужно написать middleware со сложной логикой и зависимостями, то его стоит реализовывать как класс:

```
// Middleware для измерения времени выполнения запроса
public class RequestTimingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<RequestTimingMiddleware> _logger;

    // RequestDelegate — это и есть "следующий middleware"
    // ILogger — зависимость, которая будет передана DI-контейнером
    public RequestTimingMiddleware(
        RequestDelegate next,
        ILogger<RequestTimingMiddleware> logger)
    {
        _next   = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        var stopwatch = Stopwatch.StartNew();
```

```
        // Передаём запрос дальше по конвейеру
        await _next(context);

        // Ответ уже сформирован — замеряем итоговое время
        stopwatch.Stop();

        _logger.LogInformation(
            "{Method} {Path} выполнен за {ElapsedMs} мс со статусом
{StatusCode}",
            context.Request.Method,
            context.Request.Path,
            stopwatch.ElapsedMilliseconds,
            context.Response.StatusCode);
    }
}
```

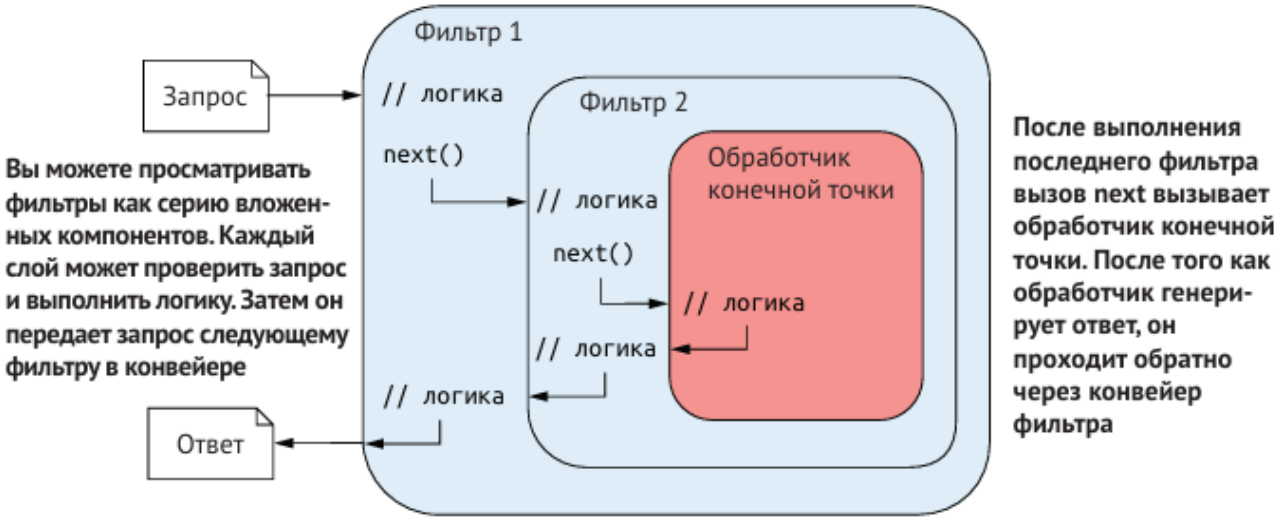
Регистрация в `Program.cs`:

```
app.UseMiddleware<RequestTimingMiddleware>();
```

3. Фильтры (Filters)

Фильтры — это специальные компоненты, которые позволяют выполнять код до или после определённых стадий обработки запроса внутри Web API. Они идеально подходят для реализации сквозной логики (авторизация, кэширование, логирование), помогая избежать дублирования кода в контроллерах. По своему принципу работы они похожи на middleware: запрос проходит через все фильтры, доходит до метода контроллера и возвращается обратно.

Принципиальное отличие от middleware состоит в том, что фильтры запускаются после того, как весь конвейер middleware отработал и запрос был направлен к конкретному контроллеру. Middleware не знает ни о контроллерах, ни о параметрах методов. Фильтры, напротив, работают уже внутри Web API и имеют доступ к контексту действия, аргументам метода и результату выполнения.



Виды фильтров и порядок их выполнения

Фильтры выполняются в строго определённом порядке:

```
Authorization → Resource → Action → Exception → Result
```

Каждый вид отвечает за свою задачу:

- **Authorization filters** — выполняются первыми, до всего остального. Если запрос не прошёл авторизацию, дальнейшие фильтры не вызываются.
- **Resource filters** — выполняются до привязки модели. Используются, например, для кэширования на уровне ресурса.
- **Action filters** — выполняются непосредственно до и после метода контроллера. Наиболее часто используемый вид.
- **Exception filters** — перехватывают необработанные исключения, возникшие в методе контроллера или в action-фильтрах. Важно понимать ограничение: они не

перехватывают исключения из middleware, из фильтров авторизации и из фильтров результата.
- **Result filters** — выполняются до и после формирования ответа из результата действия (`IActionResult`).

Встроенные фильтры

ASP.NET Web API поставляется с набором готовых фильтров, которые покрывают большинство типовых задач. Рассмотрим наиболее часто используемые.

Авторизация: `[Authorize]` и `[AllowAnonymous]`

`[Authorize]` — один из самых распространённых фильтров. Он требует, чтобы пользователь был аутентифицирован. Если запрос приходит без действительного токена или сессии, фильтр немедленно возвращает `401 Unauthorized` , не допуская выполнения метода контроллера.

```
// Весь контроллер защищён — каждый метод требует аутентификации
[Authorize]
public class ProductsController : ControllerBase
{
    // Доступен только аутентифицированным пользователям
    public IActionResult GetAll() { ... }

    // [AllowAnonymous] переопределяет [Authorize] на уровне контроллера —
    // этот метод будет доступен всем, даже без аутентификации
    [AllowAnonymous]
    public IActionResult GetFeatured() { ... }
}
```

Можно ограничить доступ по роли или политике:

```
// Доступ только пользователям с ролью Admin
[Authorize(Roles = "Admin")]
public IActionResult Delete(int id) { ... }

// Доступ по именованной политике, настроенной в Program.cs
[Authorize(Policy = "MinimumAge18")]
public IActionResult GetAdultContent() { ... }
```

Если `[Authorize]` нужен для большинства эндпоинтов, удобнее зарегистрировать его глобально, а на публичные методы явно добавлять `[AllowAnonymous]` :

```
// Program.cs — [Authorize] применяется ко всему приложению
builder.Services.AddControllers(options =>
{
```

```
    options.Filters.Add(new AuthorizeFilter());
});
```

Кэширование ответа: `[ResponseCache]`

`[ResponseCache]` управляет HTTP-кэшированием на уровне клиента и промежуточных прокси. Фильтр добавляет в ответ заголовок `Cache-Control`, указывая, как долго клиент может использовать кэшированный результат без повторного обращения к серверу:

```
// Ответ будет кэшироваться клиентом на 60 секунд
[HttpGet]
[ResponseCache(Duration = 60)]
public IActionResult GetAll() { ... }

// Кэширование отключено — ответ никогда не сохраняется в кэше
// Используется для данных, которые всегда должны быть актуальными
[HttpGet("{id}")]
[ResponseCache(NoStore = true)]
public IActionResult GetById(int id) { ... }
```

Важно понимать, что `[ResponseCache]` только выставляет заголовки — само хранение кэша на стороне сервера этот фильтр не обеспечивает. Для серверного кэширования используется отдельный middleware `UseResponseCaching`.

Документирование ответов: `[Produces]` и `[ProducesResponseType]`

`[Produces]` указывает, в каком формате контроллер возвращает данные, если клиент запросит формат, отличный от указанного, он получит `406 Not Acceptable`:

```
// Все методы контроллера возвращают только JSON
[Produces("application/json")]
public class ProductsController : ControllerBase { ... }
```

`[ProducesResponseType]` документирует возможные HTTP-статусы ответа. Сам по себе он не влияет на логику выполнения, но критически важен для автоматической генерации документации Swagger/OpenAPI — без него Swagger не знает, какие коды ответа и модели ожидать от метода:

```
[HttpGet("{id}")]
[ProducesResponseType(typeof(Product), StatusCodes.Status200OK)]    //
возвращает Product при успехе
[ProducesResponseType(StatusCodes.Status404NotFound)]               // если
продукт не найден
[ProducesResponseType(StatusCodes.Status400BadRequest)]             // если
id некорректен
```

```
public IActionResult GetById(int id)
{
    if (id <= 0)
        return BadRequest();

    var product = _repository.GetById(id);
    if (product is null)
        return NotFound();

    return Ok(product);
}
```

Собственные фильтры

Встроенных фильтров достаточно для большинства задач, однако иногда требуется логика, специфичная для конкретного приложения. В таких случаях можно реализовать собственный фильтр, внедрив нужный интерфейс — например, `IAsyncActionFilter` для логики вокруг метода контроллера или `IExceptionFilter` для централизованной обработки ошибок.

```
// Пример: фильтр, который логирует все входящие запросы к API
public class RequestLoggingFilter : IAsyncActionFilter
{
    private readonly ILogger<RequestLoggingFilter> _logger;

    public RequestLoggingFilter(ILogger<RequestLoggingFilter> logger)
    {
        _logger = logger;
    }

    public async Task OnActionExecutionAsync(
        ActionExecutingContext context,
        ActionExecutionDelegate next)
    {
        // Логируем до выполнения метода контроллера
        _logger.LogInformation(
            "Запрос: {Method} {Path}",
            context.HttpContext.Request.Method,
            context.HttpContext.Request.Path);

        var executedContext = await next(); // выполняем метод контроллера

        // Логируем после — статус ответа уже известен
        _logger.LogInformation(
            "Ответ: {StatusCode}",
            executedContext.HttpContext.Response.StatusCode);
    }
}
```

Собственный фильтр можно применить точечно через `[ServiceFilter]` или `[TypeFilter]` на конкретном методе или контроллере, либо зарегистрировать глобально через `AddControllers`:

```
// Глобальная регистрация — фильтр применяется ко всему приложению
builder.Services.AddControllers(options =>
{
    options.Filters.Add<RequestLoggingFilter>();
});
```

4. Обработка ошибок

В Web API важно не просто поймать исключение, но и вернуть клиенту понятный, структурированный ответ. Бесконтрольно выброшенное исключение либо вернёт пустой `500 Internal Server Error`, либо — что хуже — вернёт клиенту полный стек-трейс с деталями внутреннего устройства приложения.

ASP.NET предоставляет два основных инструмента для централизованной обработки ошибок: middleware `UseExceptionHandler` и стандартный формат ответа `ProblemDetails`.

ProblemDetails

`ProblemDetails` — это стандартный формат описания ошибок для HTTP API. Вместо того чтобы каждый раз придумывать собственную структуру ответа об ошибке, ASP.NET предлагает готовый формат:

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.4",
  "title": "Not Found",
  "status": 404,
  "detail": "Продукт с идентификатором 42 не найден",
  "traceId": "00-84b7b3a3c2e...-00"
}
```

Поле `traceId` особенно полезно: оно позволяет связать ответ клиента с конкретной записью в логах на сервере.

Чтобы включить автоматическое формирование таких ответов, достаточно добавить в `Program.cs`:

```
builder.Services.AddProblemDetails();
```

После этого стандартные ошибки (например, `404` при обращении к несуществующему маршруту) автоматически будут возвращаться в формате `ProblemDetails`.

UseExceptionHandler

`UseExceptionHandler` — это middleware, который перехватывает все необработанные исключения, которые не были пойманы ни фильтрами, ни другим кодом. Это последний рубеж обороны: если исключение дошло сюда, значит, никто другой его не обработал.

```
app.UseExceptionHandler(errorApp =>
{
    errorApp.Run(async context =>
    {
        // Получаем информацию о перехваченном исключении
        var exceptionFeature =
context.Features.Get<IExceptionHandlerFeature>();
        var exception = exceptionFeature?.Error;

        var logger =
context.RequestServices.GetRequiredService<ILogger<Program>>();
        logger.LogError(exception, "Необработанное исключение");

        context.Response.StatusCode = 500;
        context.Response.ContentType = "application/json";

        // Возвращаем ответ в формате ProblemDetails
        await context.Response.WriteAsJsonAsync(new
        {
            type = "https://tools.ietf.org/html/rfc7231#section-6.6.1",
            title = "Internal Server Error",
            status = 500,
            detail = "Произошла непредвиденная ошибка. Пожалуйста,
попробуйте позже."
        });
    });
});
```

Клиенту намеренно возвращается общее сообщение без деталей исключения. Реальная информация об ошибке записывается в лог и остаётся на сервере.

```
// Program.cs — итоговая конфигурация обработки ошибок

builder.Services.AddProblemDetails();

// ...

// UseExceptionHandler должен быть первым в конвейере,
// чтобы перехватывать исключения из всех последующих middleware
app.UseExceptionHandler();
```

При вызове `app.UseExceptionHandler()` без аргументов и при наличии `AddProblemDetails()` ASP.NET автоматически формирует ответ в формате `ProblemDetails` — писать собственный обработчик вручную не нужно. Ручная реализация, показанная выше, нужна только тогда, когда требуется нестандартное поведение: например, логирование с дополнительным контекстом или особый формат ответа.